

System Overview

We design a **layered, enterprise-grade** authentication/authorization system in Go (Gin) following clean-architecture principles. The system supports a super-admin (from `.env`) and regular users (with roles/permissions stored in the database). We use JWTs (with short-lived access tokens and long-lived refresh tokens) plus **HttpOnly cookies** for maximum security. We enforce RBAC with **Casbin**, log all actions for auditing, and protect against OWASP threats (e.g. brute force, XSS, etc.) with middlewares.

- **High-Level Flow:** Clients authenticate via POST `/api/v1/auth/login` (sending email/password). On success we issue an **access token** (short expiry, used for API calls) and a **refresh token** (longer expiry) – storing at least the refresh token in an HttpOnly cookie to mitigate XSS ¹. We enable **refresh-token rotation** so old refresh tokens are invalidated once used ² ³. Each JWT carries a unique ID (“jti”) so that we can revoke it if needed ³.
- **RBAC:** We use [Casbin](#) to enforce per-API permissions. Roles and module/action permissions are stored in GORM models (Role, Permission). Casbin loads policies from our DB. As per the Casbin docs, “Casbin implements RBAC with support for role hierarchies and domains” ⁴. The special “Super_Admin” role (from `.env`) bypasses all checks.
- **Security:** We add standard HTTP hardening (CSP, X-Frame-Options, etc.) via middleware ⁵ ⁶. CORS is locked to our frontend (with credentials enabled). We rate-limit login attempts to thwart brute-force attacks ⁷ ⁸. We never reveal detailed auth errors – all failures return a generic “invalid credentials” message to avoid user enumeration ⁹. All passwords are bcrypt-hashed.
- **Logging & Auditing:** We use Uber’s Zap logger for structured logs, and an **AuditLog** GORM model to record every login/logout (and later, user CRUD) action with user, module, action, IP, etc. Auditing is crucial for compliance and forensics.

Below we outline the **directory layout** and show key code snippets.

Directory Structure

We follow the given layout, with these key components (only login-related files shown here):

```
backend/
├── cmd/server/main.go      # entrypoint
├── internal/
│   ├── config/config.go    # load .env into AppConfig (using joho/godotenv or
│   │                       Viper)
│   ├── database/
│   │   ├── postgres.go     # GORM DB connection and pool
│   │   └── migrate.go      # Auto-migrate models (RunMigrations)
│   ├── models/             # GORM models (Role, Permission, User, AuditLog)
│   ├── auth/               # JWT/password utilities
│   └── jwt.go              # Token generation & parsing
```

```

├── password.go      # bcrypt hash and verify
├── domain/
│   ├── user/
│   │   ├── entity.go      # (Optional) user domain model
│   │   ├── repository.go  # IUserRepository interface
│   │   └── service.go     # IUserService interface
│   ├── repository/       # DB implementations
│   │   └── user_repo.go   # implements IUserRepository
│   ├── services/         # Business logic
│   │   └── user_service.go # implements IUserService
│   ├── handlers/         # HTTP handlers
│   │   └── auth_handler.go # Login endpoint
│   ├── middleware/       # HTTP middlewares
│   │   ├── jwt_auth.go    # JWTAuth middleware
│   │   ├── rbac.go        # RBAC middleware (Casbin)
│   │   ├── audit.go       # Audit logging middleware
│   │   ├── rate_limit.go  # Rate limiting (e.g. using golang.org/x/time/rate)
│   │   └── headers.go     # Security headers middleware
│   ├── rbac/             # Casbin setup (enforcer init with GORM adapter)
│   ├── cache/            # Redis client (for storing refresh token JTIs or
sessions)
│   ├── logger/           # Zap logger init
│   ├── routes/
│   │   ├── routes.go     # RegisterRoutes (API groups v1/v2)
│   │   └── v1/auth.go    # /api/v1/auth routes
│   └── ...
├── pkg/
│   └── response/         # APIResponse wrapper for consistent JSON replies
├── scripts/
│   └── seed.go           # Seeds (e.g. create default roles, super-admin, org)
├── .env
├── go.mod
└── go.sum

```

Each package is kept small and focused. For example, `domain/user/service.go` defines an `IUserService` interface (with a `Login(email, password)` method), which is implemented by `services/user_service.go`. The handler (`auth_handler.go`) depends only on the service interface, keeping handlers thin and testable.

Configuration

Our `config.Config` struct (in `internal/config/config.go`) reads `.env` variables. Key settings include the JWT secret and expiration, Postgres/Redis credentials, and the built-in Super Admin credentials. For example:

```

type AppConfig struct {
    JWT      JWTConfig
    DB        DatabaseConfig
    Redis     RedisConfig
    // ...
    SuperAdmin SuperAdmin
}
type JWTConfig struct {
    Secret      string
    AccessTTL   time.Duration // in minutes
    RefreshTTL  time.Duration
}

```

We load these (using `godotenv` or a similar library) on startup. Critically, `SUPER_ADMIN_EMAIL` and `SUPER_ADMIN_PASSWORD` come from `.env`. This ensures **Super_Admin** is entirely defined by our deployment – it is not in the database and cannot be deleted or altered at runtime. (By default we set `IsSystem=true` on this role so migrations/seed won't allow deletion.)

Data Models

We use GORM models for the database schema:

```

// models/base.go
type BaseModel struct {
    ID          uuid.UUID `gorm:"type:uuid;primaryKey;default:gen_random_uuid()"`
    CreatedAt   time.Time
    UpdatedAt   time.Time
    DeletedAt   gorm.DeletedAt `gorm:"index"`
}

// models/role.go
type Role struct {
    BaseModel
    Name          string `gorm:"uniqueIndex;not null"`
    Description    string
    IsActive      bool   `gorm:"default:true"`
    IsSystem      bool   `gorm:"default:false" // true for built-in roles (e.g.
Super_Admin)`
    Permissions []Permission
    `gorm:"many2many:role_permissions;constraint:OnDelete:CASCADE"`
}

// models/permission.go
type Permission struct {
    BaseModel

```

```

Module string `gorm:"not null;index:idx_module_action,unique"`
Action string `gorm:"not null;index:idx_module_action,unique"`
}

// models/user.go
type User struct {
    BaseModel
    OrganizationID uuid.UUID
    BranchID       *uuid.UUID
    RoleID         uuid.UUID
    Name           string
    Email          string `gorm:"uniqueIndex;not null"`
    Phone          string `gorm:"size:20"`
    PasswordHash   string `json:"-` // do not JSON-serialize
    Status         string `gorm:"type:varchar(20);default:'active'"`
    LastLoginAt    *time.Time
    JoiningDate    time.Time
    // personal fields ...
    Role           Role `gorm:"foreignKey:RoleID"`
}

// models/auditlog.go
type AuditLog struct {
    BaseModel
    UserID      uuid.UUID `gorm:"type:uuid;index"`
    Action      string    `gorm:"size:50;not null" // e.g. LOGIN, CREATE, UPDATE
    Module      string    `gorm:"size:50;not null" // e.g. auth, users, inventory
    IP          string    `gorm:"size:45"`
    UserAgent   string    `gorm:"size:255"`
    Details     string    `gorm:"type:text" // optional JSON or info
}

```

- **Role-Permission:** Many-to-many via `role_permissions` table (GORM handles pivot). Each permission has a unique (Module,Action) pair (e.g. `users:read`, `users:manage`). The `"manage"` action is shorthand for all CRUD on that module.
- **User:** Belongs to one Role. The `PasswordHash` stores the bcrypt hash. `Status` can be `"active"` or `"inactive"`. We preload the `Role` on login to get role name/permissions.
- **Super_Admin:** This is represented by a config, not a `User` record. We treat `UserID = uuid.Nil` and `Role.Name = "Super_Admin"` for this special case in code.

We run `database.DB.AutoMigrate(&Role{}, &Permission{}, &User{}, &AuditLog{}, ...)` at startup to create the tables.

Authentication Service

We implement an `IUserService` in the domain layer:

```

type IUserService interface {
    // Validate credentials and return user (or error)
    Authenticate(email, password string) (*models.User, error)
    // (In future: user CRUD methods)
}

```

In `services/user_service.go`, we inject an `IUserRepository` and the `config.Cfg.JWT` :

```

type userService struct {
    repo domain.IUserRepository
    jwtCfg config.JWTConfig
}

func NewUserService(repo domain.IUserRepository, cfg config.JWTConfig)
domain.IUserService {
    return &userService{repo: repo, jwtCfg: cfg}
}

func (s *userService) Authenticate(email, password string) (*models.User,
error) {
    // 1) Lookup user by email
    user, err := s.repo.FindByEmail(email)
    if err != nil {
        return nil, errors.New("invalid credentials")
    }
    // 2) Check password (bcrypt)
    if !auth.CheckPassword(user.PasswordHash, password) {
        // Generic error to avoid user enumeration 9
        return nil, errors.New("invalid credentials")
    }
    // 3) Check active status
    if user.Status != "active" {
        return nil, errors.New("account inactive")
    }
    return user, nil
}

```

- **Password Verification:** We use a bcrypt wrapper (e.g. `auth.HashPassword` and `auth.CheckPassword`), never storing plaintext.
- **Generic Errors:** We always return a generic error (e.g. "invalid credentials") so the client cannot infer whether it was the email or password that was wrong ⁹. This thwarts username enumeration and timing attacks.
- **Repository:** The `FindByEmail` method in `repository/user_repo.go` simply does `db.Where("email = ?", email).First(&user)` with GORM.

JWT Generation (auth/jwt.go)

We use HMAC-SHA256 (HS256) for JWTs. We define two functions: `GenerateAccessToken(userID, role)` and `GenerateRefreshToken(userID)`. Each token includes a unique JWT ID (`jti`) so we can revoke it if needed ³. Pseudocode:

```
// Claims for access token
type JWTClaims struct {
    UserID uuid.UUID `json:"user_id"`
    Role   string   `json:"role"`
    jwt.RegisteredClaims
}

// GenerateAccessToken returns a signed JWT string (expires after accessTTL
minutes).
func GenerateAccessToken(userID uuid.UUID, role, secret string, ttlMin int)
(string, error) {
    now := time.Now()
    claims := JWTClaims{
        UserID: userID,
        Role:   role,
        RegisteredClaims: jwt.RegisteredClaims{
            ExpiresAt:
jwt.NewNumericDate(now.Add(time.Duration(ttlMin)*time.Minute)),
            IssuedAt:  jwt.NewNumericDate(now),
            ID:       uuid.NewString(), // Unique jti for revocation 3
            Subject:  userID.String(),
        },
    }
    token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
    return token.SignedString([]byte(secret))
}

// GenerateRefreshToken returns a signed JWT refresh token (longer TTL).
func GenerateRefreshToken(userID uuid.UUID, secret string, ttlMin int) (string,
error) {
    now := time.Now()
    claims := jwt.RegisteredClaims{
        ExpiresAt:
jwt.NewNumericDate(now.Add(time.Duration(ttlMin)*time.Minute)),
        IssuedAt:  jwt.NewNumericDate(now),
        ID:       uuid.NewString(),
        Subject:  userID.String(), // link to user
    }
    token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
    return token.SignedString([]byte(secret))
}
```

```

}

// ValidateToken parses and validates a JWT string.
func ValidateToken(tokenStr, secret string) (*JWTClaims, error) {
    token, err := jwt.ParseWithClaims(tokenStr, &JWTClaims{}, func(t
*jwt.Token) (interface{}, error) {
        if _, ok := t.Method.(*jwt.SigningMethodHMAC); !ok {
            return nil, errors.New("unexpected signing method")
        }
        return []byte(secret), nil
    })
    if err != nil {
        return nil, err
    }
    claims := token.Claims.(*JWTClaims)
    return claims, nil
}

```

Note:

- We set `ID` (the "jti") to a UUID to uniquely identify each token ³.
- **Revocation:** On logout or token rotation, we can store the `jti` in Redis or DB to mark it invalid. (The Vonage tutorial shows using Redis to store token identifiers and TTL ¹⁰.) We set the refresh token's `Subject` to the `userID` and can tie that to the access token's `jti`.
- **Cookie Storage:** We will set at least the refresh token in an `HttpOnly`, `Secure` cookie (`SameSite=Lax`) to protect against XSS ¹. The access token may also be set in a cookie or returned in JSON.

Middleware

- **JWTAuth:** Verifies the `Authorization: Bearer <token>` header. On success, it populates the Gin context with `user_id` and `role` so handlers/middleware can see them. If missing or invalid, return `401 Unauthorized`.
- **RBAC:** After JWTAuth, we enforce permissions with Casbin. For example `RBAC("users", "read")` middleware ensures the user's role has the `"users", "read"` permission. Casbin's policy (stored e.g. in MySQL/Postgres via Gorm-adaptor) checks role-to-permission mappings. Our special "Super_Admin" role automatically passes all checks.
- **Security Headers:** We add common HTTP headers on every response (middleware). For example:

```

c.Header("X-Content-Type-Options", "nosniff")
c.Header("X-Frame-Options", "DENY")
c.Header("Content-Security-Policy", "default-src 'self'; ...")
c.Header("Strict-Transport-Security",
    "max-age=31536000; includeSubDomains; preload")

```

This follows OWASP recommendations to prevent XSS, clickjacking, MIME-sniffing, etc. ⁵ ⁶ .

- **CORS:** We use Gin's CORS middleware (github.com/gin-contrib/cors) with `AllowCredentials: true`, `AllowOrigins` set from `FRONTEND_URL`, and specific methods/headers. We **never** use `"*"` when cookies are involved ¹¹ .
- **Rate Limiting:** We protect the `/login` endpoint with a rate limiter (e.g. [x/time/rate](#) or a token bucket). This defends against credential-stuffing and brute-force. OWASP suggests limiting failed attempts or delaying responses ⁸ ⁷ . In practice, we might allow e.g. 5 attempts per minute per IP. (Production systems might also lock accounts temporarily.)
- **Audit Logging:** We record every login (and other sensitive action) in the `audit_logs` table. We implement an `AuditMiddleware` that logs user ID, route, action, IP, user-agent, etc. For example, on successful login we insert an `AuditLog{UserID, Action:"LOGIN", Module:"auth", IP, UserAgent}`. Keeping detailed audit trails is best practice for compliance (see [audit best practices](#)).

Handler and Routes

In `internal/handlers/auth_handler.go` we implement the `Login` handler:

```
type LoginRequest struct {
    Email    string `json:"email" binding:"required,email"`
    Password string `json:"password" binding:"required"`
}

type LoginResponseData struct {
    AccessToken string `json:"access_token"`
    RefreshToken string `json:"refresh_token"`
    ExpiresAt    time.Time `json:"expires_at"`
    User         struct {
        ID      uuid.UUID `json:"id"`
        Name    string    `json:"name"`
        Email   string    `json:"email"`
        Role    string    `json:"role"`
    } `json:"user"`
}

func LoginHandler(c *gin.Context) {
    var req LoginRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        res := response.NewErrorResponse("INVALID_REQUEST", "Invalid JSON", nil)
        c.JSON(http.StatusBadRequest, res)
        return
    }

    // 1. Check if Super Admin
    if req.Email == config.Cfg.SuperAdmin.Email {
        if req.Password != config.Cfg.SuperAdmin.Password {
```



```

        c.JSON(http.StatusUnauthorized, response.NewErrorResponse("UNAUTH",
"Invalid credentials", nil))
        return
    }
    // Create tokens (userID=Nil, role=Super_Admin)
    accessToken, _ := auth.GenerateAccessToken(uuid.Nil, "Super_Admin",
config.Cfg.JWT.Secret, config.Cfg.JWT.AccessTTL)
    refreshToken, _ := auth.GenerateRefreshToken(uuid.Nil,
config.Cfg.JWT.Secret, config.Cfg.JWT.RefreshTTL)
    expiresAt := time.Now().Add(time.Duration(config.Cfg.JWT.AccessTTL) *
time.Minute)

    // Set refresh token in HttpOnly cookie (optional)
    c.SetCookie("refresh_token", refreshToken,
int(config.Cfg.JWT.RefreshTTL*60), "/", "", true, true)

    // Respond
    respData := LoginResponseData{
        AccessToken: accessToken,
        RefreshToken: refreshToken,
        ExpiresAt:    expiresAt,
    }
    respData.User.ID = uuid.Nil
    respData.User.Name = "Super Admin"
    respData.User.Email = config.Cfg.SuperAdmin.Email
    respData.User.Role = "Super_Admin"

    c.JSON(http.StatusOK, gin.H{
        "success": true,
        "isAuthenticated": true,
        "message": "login successful",
        "data": respData,
    })
    // Audit log
    go func() {
        database.DB.Create(&models.AuditLog{
            UserID:    uuid.Nil,
            Action:    "LOGIN",
            Module:    "auth",
            IP:        c.ClientIP(),
            UserAgent: c.Request.UserAgent(),
            Details:    "Super_Admin login",
        })
    }()
    return
}

// 2. Normal user login

```

```

user, err := userService.Authenticate(req.Email, req.Password)
if err != nil {
    // Always use generic error message 9
    c.JSON(http.StatusUnauthorized, gin.H{
        "success": false, "message": "Invalid email or password",
    })
    return
}

// 3. Generate JWTs
accessToken, _ := auth.GenerateAccessToken(user.ID, user.Role.Name,
config.Cfg.JWT.Secret, config.Cfg.JWT.AccessTTL)
refreshToken, _ := auth.GenerateRefreshToken(user.ID,
config.Cfg.JWT.Secret, config.Cfg.JWT.RefreshTTL)
expiresAt := time.Now().Add(time.Duration(config.Cfg.JWT.AccessTTL) *
time.Minute)
c.SetCookie("refresh_token", refreshToken,
int(config.Cfg.JWT.RefreshTTL*60), "/", "", true, true)

// 4. Update last login timestamp
now := time.Now()
database.DB.Model(&user).Update("last_login_at", now)

// 5. Audit log
go func() {
    database.DB.Create(&models.AuditLog{
        UserID:    user.ID,
        Action:     "LOGIN",
        Module:    "auth",
        IP:        c.ClientIP(),
        UserAgent: c.Request.UserAgent(),
        Details:    "",
    })
}()

// 6. Return success
respData := LoginResponseData{
    AccessToken: accessToken,
    RefreshToken: refreshToken,
    ExpiresAt:    expiresAt,
}
respData.User.ID = user.ID
respData.User.Name = user.Name
respData.User.Email = user.Email
respData.User.Role = user.Role.Name

c.JSON(http.StatusOK, gin.H{
    "success": true,

```

```

        "isAuthenticated": true,
        "message":         "login successful",
        "data":            respData,
    })
}

```

Notes:

- We parse JSON and validate it. We rely on Gin's `binding: "required,email"` to catch bad input.
- We handle the **Super_Admin** case first, using credentials from `.env`. This user's `user_id` is `uuid.Nil` and their role is `"Super_Admin"`. We skip the DB entirely for this login.
- For normal users, we call the service to authenticate. On failure we return `401` with a generic message ("Invalid email or password") ⁹.
- On success, we generate tokens. We *both* include the tokens in JSON **and** set the refresh token in an `HttpOnly` cookie. (This gives flexibility: the client can read the JSON `access_token`, or we could alternate and put that in a cookie too.)
- We include `expires_at` and basic user info (id, name, email, role) in the response, as per example.
- **Audit:** We asynchronously insert a row into `audit_logs`. This ensures we record the login event without delaying the HTTP response.
- **Cookie Settings:** `c.SetCookie("refresh_token", ...)` uses `Secure=true` (so HTTPS only) and `HttpOnly=true` by default (Gin does this when `raw` is true). We can also set `SameSite=Lax` by manually adding a header if needed.
- If the user's account is inactive, we *could* return 403 Forbidden. In the example above we simply returned "invalid credentials" to avoid revealing status. A more friendly approach might say "account inactive" but only after verifying existence – this is a design choice.

We register this handler in `routes/v1/auth.go`:

```

func RegisterAuthRoutes(rg *gin.RouterGroup) {
    auth := rg.Group("/auth")
    {
        auth.POST("/login", handlers.LoginHandler)
        // (Optionally) auth.POST("/refresh", handlers.RefreshHandler)
        // auth.POST("/logout", handlers.LogoutHandler)
    }
}

```

And in `routes/routes.go`:

```

func RegisterRoutes(r *gin.Engine) {
    apiV1 := r.Group("/api/v1")
    routes_v1 := apiV1.Group("/")
    {
        routes_v1.Use(/* global v1 middleware */)
        routes_v1.Use(RegisterAuthRoutes)
    }
}

```

```

        // other v1 routes...
    }
    // future v2...
}

```

Middleware: JWT Auth & RBAC

The `JWTAuth` middleware (in `internal/middleware/jwt_auth.go`) extracts and validates the access token on every protected endpoint:

```

func JWTAuthMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        authHeader := c.GetHeader("Authorization")
        parts := strings.SplitN(authHeader, " ", 2)
        if len(parts) != 2 || strings.ToLower(parts[0]) != "bearer" {
            c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error":
"missing or invalid token"})
            return
        }
        tokenStr := parts[1]
        claims, err := auth.ValidateToken(tokenStr, config.Cfg.JWT.Secret)
        if err != nil {
            c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error":
"invalid or expired token"})
            return
        }
        // Set context values
        c.Set("user_id", claims.UserID.String())
        c.Set("role", claims.Role)
        c.Next()
    }
}

```

The `RBAC` middleware (in `internal/middleware/rbac.go`) calls Casbin's `Enforce` for the required module/action:

```

func RBACMiddleware(module, action string) gin.HandlerFunc {
    return func(c *gin.Context) {
        roleName := c.GetString("role")
        if roleName == "" {
            c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error":
"unauthorized"})
            return
        }
        if !rbac.Enforce(roleName, module, action) {

```

```

        c.AbortWithStatusJSON(http.StatusForbidden, gin.H{"error": "access
denied"})
        return
    }
    c.Next()
}
}

```

Casbin is configured in `internal/rbac/casbin.go` to use our GORM policy adapter and model. We load policies at startup. For example, a CSV or DB policy might have lines like `p, admin, users, read` meaning the *admin* role can read the *users* resource. The `Super_Admin` role is typically given a wildcard policy or we hard-code it to always succeed.

Token Storage and Revocation

As recommended, we store refresh tokens in an `HttpOnly` secure cookie, which is not accessible to JavaScript ¹. This mitigates XSS. We also use **refresh-token rotation**: whenever the client calls a `/refresh` endpoint to get a new access token, we issue a new refresh token and delete (or blacklist) the old one. We can keep a whitelist of valid refresh-token JTIs in Redis, so that once a refresh token is used or expires, its JTI is deleted ¹⁰. This ensures stolen refresh tokens cannot be reused.

For example, on `/refresh` we would do: validate old refresh token (checking its JTI in Redis), then generate new pair, store the new JTI and remove the old. In `JWTAuth` for access tokens, we could check a Redis blacklist to see if the token's JTI has been revoked (e.g. on logout).

This pattern follows the Vonage tutorial on secure JWT in Go: “store their jti (unique ID) in Redis with a TTL... we can revoke them instantly at logout, rotate refresh tokens, [and] keep multiple device sessions” ¹⁰.

Rate Limiting

To guard against brute-force login attempts, we add rate limiting middleware on the `/login` endpoint (and possibly others like `/auth/refresh`). We might use an in-memory token bucket (for small scale) or a Redis-based limiter for distributed setups. For example, allow **5 login attempts per minute per IP**. If exceeded, return `429 Too Many Requests`. This aligns with OWASP's advice to lock accounts or slow down attackers after repeated failures ⁸ ⁷.

The OneUptime blog sums it up: “Protect against brute force and DoS attacks” by rate limiting requests ⁷.

API Responses

We return JSON with a consistent structure. For successful login, the JSON (per example) is:

```

{
  "success": true,

```

```

    "isAuthenticated": true,
    "message": "login successful",
    "data": {
      "access_token": "...",
      "refresh_token": "...",
      "expires_at": "2026-04-24T15:26:23.5766274+06:00",
      "user": {
        "id": "00000000-0000-0000-0000-000000000000",
        "name": "Super Admin",
        "email": "superadmin@pharmaciano.com",
        "role": "Super_Admin"
      }
    }
  }
}

```

For errors, we use a standard error format, e.g.:

```

{
  "success": false,
  "error": {
    "code": "UNAUTH",
    "message": "Invalid credentials"
  }
}

```

(Our `pkg/response` helpers generate these wrappers.) We ensure HTTP status codes match the error (e.g. 401 for unauthorized, 400 for bad request).

Logging and Monitoring

We use [Zap](#) for structured logging (`internal/logger/zap.go` sets up a global `logger.Log` instance). We log each HTTP request in Gin's logger (for access logs) and use `logger.Log.Info/Error` in code. Important actions (login failures, rate-limit hits, etc.) should be logged with details (without sensitive info).

Audit logs in the DB provide an immutable trail of who did what and when (especially useful for forensics and compliance).

Testing the Login Endpoint

Start the server (`go run cmd/server/main.go`). Then to test Super Admin login:

```
curl -i -X POST http://localhost:8080/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"superadmin@pharmaciano.com","password":"superAdminPass123"}'
```

Replace the password with `SUPER_ADMIN_PASSWORD` from your `.env`. A successful response should have `200 OK` and JSON as above. You will also see a `Set-Cookie: refresh_token=...; HttpOnly; Secure; Path=/; Max-Age=...` header for the refresh token.

For a normal user (after seeding a user), use their email/password. On failure, you should get `401 Unauthorized` with a generic error:

```
{
  "success": false,
  "message": "Invalid email or password"
}
```

Further Considerations and Questions

To make this system even stronger, consider the following questions:

- **Multi-Factor Authentication (MFA):** Should we require a second factor (e.g. TOTP, SMS code) for login, especially for admin roles? MFA is one of the best defenses against credential compromise ⁹.
- **Key Management:** How are we rotating the JWT signing secret? If the key is compromised, all tokens become invalid. We might want to support key rotation or use asymmetric keys (RS256) for better security.
- **Refresh Token Storage:** Currently we use cookies + Redis. Should we bind refresh tokens to a device or fingerprint? Should we allow concurrent sessions, and how to revoke all at once?
- **CSRF Protection:** We're using cookies for refresh tokens. We should ensure CSRF tokens on state-changing endpoints (or use SameSite cookies) to avoid token misuse.
- **Password Policies:** What are our rules for password strength, expiration, or history? Do we enforce account lockouts after repeated failures?
- **Brute-force Logging:** Besides rate limiting, do we monitor logs for suspicious patterns (multiple IPs hitting the same account)?
- **Testing & Monitoring:** Perform regular penetration testing and use a WAF to filter known attacks. Log and alert on anomalies (e.g. many 401s).
- **Zero Trust / Least Privilege:** Are we granting only necessary permissions to each role? Periodically audit roles and permissions.
- **Data Encryption:** Ensure sensitive fields (e.g. NID, address) are handled securely. Use TLS everywhere.

Each of these questions would be followed up with design choices and potentially new implementation.

Sources: Best practices in JWT auth and API security ¹ ⁵ ³ ⁹, OWASP guidelines on brute force and error handling ⁸ ⁹, and Casbin RBAC documentation ⁴ have informed this design.

1 2 10 **JWT Authentication in Go with Gin**

<https://developer.vonage.com/en/blog/using-jwt-for-authentication-in-a-golang-application-dr>

3 5 6 7 11 **How to Secure Go APIs Against OWASP Top 10**

<https://oneuptime.com/blog/post/2026-01-07-go-secure-apis-owasp-top-10/view>

4 **RBAC Overview | Apache Casbin (Incubating)**

<https://casbin.apache.org/docs/rbac-overview/>

8 **Blocking Brute Force Attacks | OWASP Foundation**

https://owasp.org/www-community/controls/Blocking_Brute_Force_Attacks

9 **Authentication - OWASP Cheat Sheet Series**

https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html